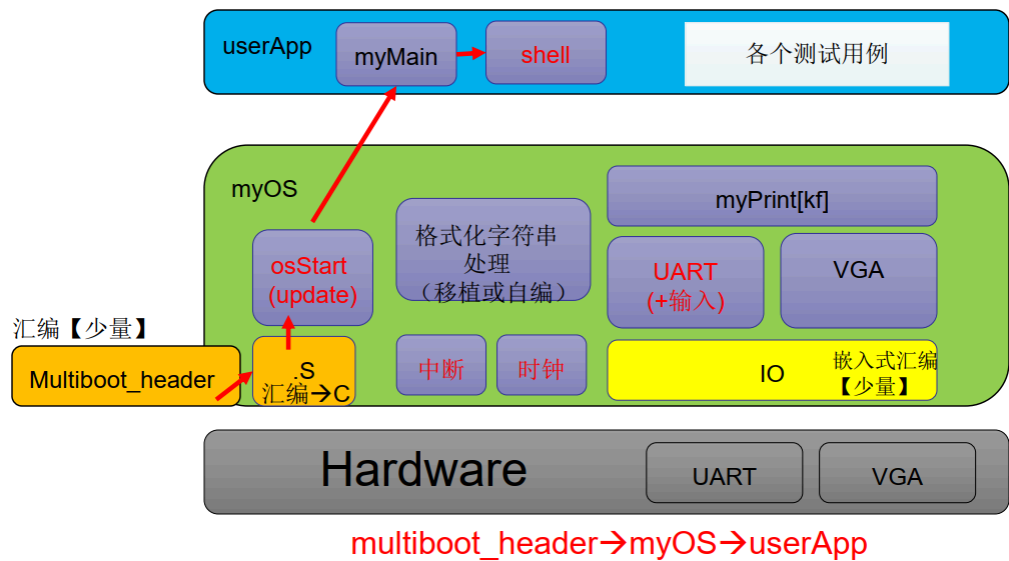


Lab2实验报告

李毅PB22051301

第一部分 系统布局



主要功能模块:

内核: 中断处理, 时钟

用户: shell

第二部分 代码解释

1.myOS/start32.S 中的 time_interrupt 和 ignore_int1 的填写

```
.p2align 4
time_interrupt:
    cld                # 将标志寄存器的方向标志位 DF 清零
    pushf              # 将标志寄存器 (Flag) 中的值存入栈中
    pusha              # 将通用寄存器 (eax, ebx 等) 中的值存入栈中
    call tick          # 调用tick函数
    popa               # 恢复通用寄存器
    popf               # 恢复标志寄存器
    iret               # 返回

.p2align 4
ignore_int1:
    #同理
    cld
    pushf
    pusha
    popa
    popf
    iret
```

2.myOS/dev/i8253.c 和 myOS/dev/i8259A.c 的填写

参考ppt, outb 函数往相应的地址输出相应的数值即可。

(1) myOS/dev/i8253.c

```
#include "io.h"

void init8253(void){
    //TODO 你需要填写它
    int f_8253 = 1193180; // 8253的时钟频率
    int f_time_interrupt = 100; // 定时中断频率
    unsigned short f_div = f_8253 / f_time_interrupt;

    outb(0x43, 0x34);
    outb(0x40, (unsigned char)f_div);
    outb(0x40, (unsigned char)(f_div >> 8));
}
```

(2) myOS/dev/i8259A.c

```
#include "io.h"

void init8259A(void){
    //TODO 你需要填写它
    //mask
    outb(0x21, 0xFF);
    outb(0xA1, 0xFF);
    //主片
    outb(0x20, 0x11);
    outb(0x21, 0x20);
    outb(0x21, 0x04);
    outb(0x21, 0x3);
    //从片
    outb(0xA0, 0x11);
    outb(0xA1, 0x28);
    outb(0xA1, 0x02);
    outb(0xA1, 0x01);
}
```

3.myOS/i386/irq.s 的填写

```
.text
.code32
_start:
    .globl enable_interrupt
enable_interrupt:
    # TODO 你需要填写它
    sti
    ret

    .globl disable_interrupt
disable_interrupt:
    # TODO 你需要填写它
    cli
```

```
ret
```

开/关中断，再返回即可

4.Tick 的实现

```
#include "wallClock.h"
int system_ticks;
int HH,MM,SS;
//NOTE:你可以自行定义接口来辅助实现

void tick(void){
    system_ticks++;
    //TODO 你需要填写它，100个时钟周期增加1秒
    if (system_ticks % 100 == 0)
    {
        SS++;
        if (SS >= 60)
        {
            SS = 0;
            MM++;
            if (MM >= 60)
            {
                MM = 0;
                HH = (HH + 1) % 24;
            }
        }
    }
    setWallClock(HH,MM,SS);
    return;
}
```

如上，直接实现即可

5.WallClock 的实现

```
#include "vga.h"
#include "wallClock.h"

//NOTE:实现了时钟设置和获取的接口，如何在tick函数中使用它们，随着时钟中断更新墙钟？
void setWallClock(int HH,int MM,int SS){
    //TODO 你需要填写它
    char time[8];//HH:MM:SS
    time[0] = HH / 10 + '0';
    time[1] = HH % 10 + '0';
    time[2] = ':';
    time[3] = MM / 10 + '0';
    time[4] = MM % 10 + '0';
    time[5] = ':';
    time[6] = SS / 10 + '0';
    time[7] = SS % 10 + '0';

    put_chars(time, 0x3, VGA_ROW - 1, VGA_COL - 8);
}
```

```

void getWallClock(int *h, int *m, int *s){
    //TODO 你需要填写它
    extern int HH, MM, SS;  // 声明在 tick.c 中定义的全局变量
    *h = HH;
    *m = MM;
    *s = SS;
}

```

如上，直接实现即可

6.Shell 的实现

```

#include "io.h"
#include "myPrintk.h"
#include "uart.h"
#include "vga.h"
#include "i8253.h"
#include "i8259A.h"
#include "tick.h"
#include "wallClock.h"
//NOTE:你可以自行定义辅助函数来帮助你实现
typedef struct myCommand {
    char name[80];
    char help_content[200];
    int (*func)(int argc, char (*argv)[8]);
}myCommand;

// Forward-declare command functions
static int func_cmd(int argc, char (*argv)[8]);
static int func_help(int argc, char (*argv)[8]);

myCommand cmd={"cmd\0","List all command\n\0",func_cmd};
myCommand help={"help\0","usage: help [command]\n\0Display info about
[command]\n\0",func_help};

static myCommand *commands[] = { &cmd, &help };
static const int cmd_count = 2;

int strcmp(char *a,char *b){
    while((*a==*b)&&*a!='\0'){a++;b++;}
    return *a-*b;
}

int func_cmd(int argc, char (*argv)[8]){
    int i;
    myPrintk(0x07, "Supported commands:\n");
    for (i = 0; i < cmd_count; i++)
    {
        // 列出命令名及简介
        uart_put_chars(commands[i]->name);
        uart_put_char('\n');
        append2screen(commands[i]->name, 0x07);
        append2screen("\n", 0x07);
    }
    return 0;
}

```

```

}

int func_help(int argc, char (*argv)[8]) {
    int i;
    if (argc == 1)
    {
        // show help for all commands
        for (i = 0; i < cmd_count; i++)
        {
            if (strcmp(commands[i]->name, "help") == 0)
            {
                uart_put_chars(commands[i]->help_content);
                uart_put_char('\n');
                append2screen(commands[i]->help_content, 0x07);
                append2screen("\n", 0x07);
                return 0;
            }
        }
    }
    else
    {
        // show help for argv[1]
        for (i = 0; i < cmd_count; i++) {
            if (strcmp(argv[1], commands[i]->name) == 0)
            {
                uart_put_chars(commands[i]->help_content);
                uart_put_char('\n');
                append2screen(commands[i]->help_content, 0x07);
                append2screen("\n", 0x07);
                return 0;
            }
        }
        append2screen("no help\n", 0x07);
    }
    return 0;
}

void startShell(void){
    //我们通过串口来实现数据的输入
    char BUF[256]; //输入缓存区
    int BUF_len=0; //输入缓存区的长度

    int argc;
    char argv[8][8];

    do{
        BUF_len=0;
        myPrintk(0x07, "Student>>\0");
        while((BUF[BUF_len]=uart_get_char())!='\r'){
            uart_put_char(BUF[BUF_len]); //将串口输入的数存入BUF数组中
            BUF_len++; //BUF数组的长度加
        }
        uart_put_chars(" -pseudo_terminal\0");
        uart_put_char('\n');
        // TODO: 你需要填写它
    }
}

```

数

```
//OK,助教已经帮助你们实现了“从串口中读取数据存储在BUF数组中”的任务，接下来你们要做
//的就是对BUF数组中存储的数据进行处理(也即，从BUF数组中提取相应的argc和argv参
//数)，再根据argc和argv，寻找相应的myCommand ***实例，进行***.func(argc,argv)函

//调用。

//比如BUF中的内容为“help cmd”
//那么此时的argc为2 argv[0]为help argv[1]为cmd
//接下来就是 help.func(argc, argv)进行函数调用即可

// 解析 BUF 到 argv
argc = 0;
int idx = 0;
while (idx < BUF_len)
{
    // 跳过空格
    while (idx < BUF_len && BUF[idx] == ' ') idx++;

    if (idx >= BUF_len) break;
    int arg_len = 0;
    // 读取一个单词，不超过 7 个字符
    while (idx < BUF_len && BUF[idx] != ' ' && arg_len < 7)
    {
        argv[argc][arg_len++] = BUF[idx++];
    }
    argv[argc][arg_len] = '\0';
    // 跳过单词剩余字符
    while (idx < BUF_len && BUF[idx] != ' ') idx++;
    argc++;
    if (argc >= 8) break;
}

// 调用命令
if (argc > 0) {
    int i;
    for (i = 0; i < cmd_count; i++)
    {
        if (strcmp(argv[0], commands[i]->name) == 0)
        {
            commands[i]->func(argc, argv);
            break;
        }
    }
    if (i == cmd_count)
    {
        uart_put_chars("Unknown command: ");
        uart_put_chars(argv[0]);
        uart_put_char('\n');
        append2screen("Unknown command\n", 0x07);
        append2screen(argv[0], 0x07);
        append2screen("\n", 0x07);
    }
}

}while(1);
```

```
}
```

核心就是字符串分割以及输出，使用uart_put_char()和 append2screen()函数分别在uart和vga上输出。

按照以下顺序定义，先声明func_cmd, func_help等函数，后直接定义全局变量myCommand以及myCommand数组。后func_cmd函数进行调用。防止出现unknown variable。

这里手搓了个strcmp函数

第三部分 编译过程进行说明

src文件夹结构如下：

```
liyi@MSI:~/OSLab/Lab3$ tree
.
├── Makefile
├── Makefile:Zone.Identifier
├── compile_flags.txt
├── compile_flags.txt:Zone.Identifier
├── multibootheader
│   ├── multibootHeader.S
│   └── multibootHeader.S:Zone.Identifier
├── myOS
│   ├── Makefile
│   ├── Makefile:Zone.Identifier
│   ├── dev
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── i8253.c
│   │   ├── i8253.c:Zone.Identifier
│   │   ├── i8259A.c
│   │   ├── i8259A.c:Zone.Identifier
│   │   ├── uart.c
│   │   ├── uart.c:Zone.Identifier
│   │   ├── vga.c
│   │   └── vga.c:Zone.Identifier
│   ├── i386
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── io.c
│   │   ├── io.c:Zone.Identifier
│   │   ├── irq.S
│   │   ├── irq.S:Zone.Identifier
│   │   ├── irqs.c
│   │   └── irqs.c:Zone.Identifier
│   ├── include
│   │   ├── i8253.h
│   │   ├── i8253.h:Zone.Identifier
│   │   ├── i8259A.h
│   │   ├── i8259A.h:Zone.Identifier
│   │   ├── io.h
│   │   ├── io.h:Zone.Identifier
│   │   ├── irqs.h
│   │   ├── irqs.h:Zone.Identifier
│   │   ├── myPrintk.h
│   │   └── myPrintk.h:Zone.Identifier
```

```

| | └─ tick.h
| | └─ tick.h:Zone.Identifier
| | └─ uart.h
| | └─ uart.h:Zone.Identifier
| | └─ vga.h
| | └─ vga.h:Zone.Identifier
| | └─ vsprintf.h
| | └─ vsprintf.h:Zone.Identifier
| | └─ wallClock.h
| | └─ wallClock.h:Zone.Identifier
| └─ kernel
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ tick.c
| | └─ tick.c:Zone.Identifier
| | └─ wallClock.c
| | └─ wallClock.c:Zone.Identifier
| └─ myOS.ld
| └─ myOS.ld:Zone.Identifier
| └─ osStart.c
| └─ osStart.c:Zone.Identifier
| └─ printk
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ myPrintk.c
| | └─ myPrintk.c:Zone.Identifier
| | └─ vsprintf.c
| | └─ vsprintf.c:Zone.Identifier
| └─ start32.S
| └─ start32.S:Zone.Identifier
└─ output
  └─ multibootheader
    └─ multibootHeader.o
  └─ myOS
    └─ dev
      └─ i8253.o
      └─ i8259A.o
      └─ uart.o
      └─ vga.o
    └─ i386
      └─ io.o
      └─ irq.o
      └─ irqs.o
    └─ kernel
      └─ tick.o
      └─ wallClock.o
    └─ osStart.o
    └─ printk
      └─ myPrintk.o
      └─ vsprintf.o
    └─ start32.o
  └─ myOS.elf
    └─ userApp
      └─ main.o
      └─ startShell.o
└─ source2run.sh

```



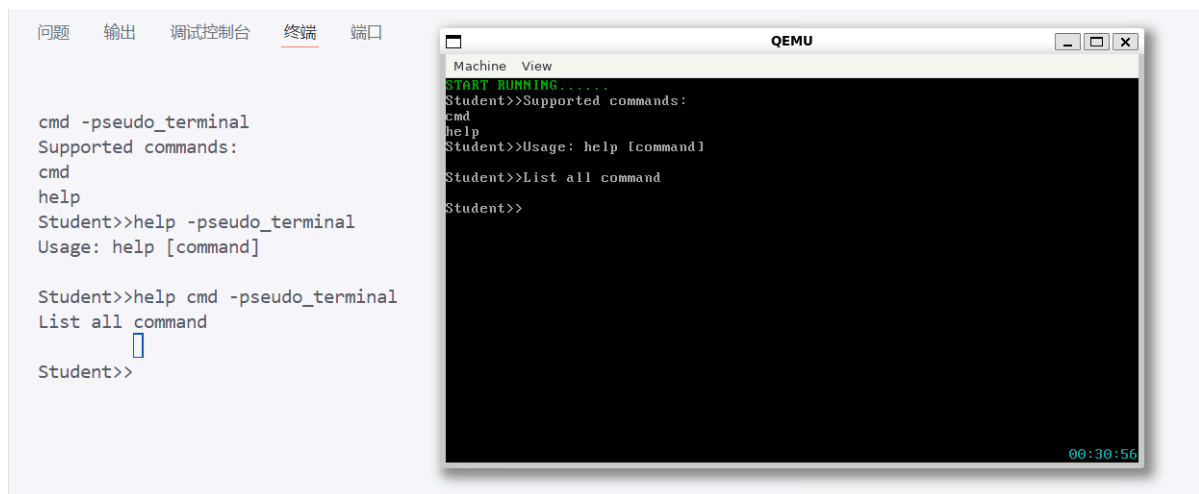
```
├─ source2run.sh:Zone.Identifier
└─ userApp
    ├─ Makefile
    │   └─ Makefile:Zone.Identifier
    ├─ main.c
    │   └─ main.c:Zone.Identifier
    ├─ startShell.c
    │   └─ startShell.c:Zone.Identifier
```

17 directories, 89 files

输入./source2run.sh 指令后，编译，链接，生成 myOS.elf 文件。输入指令 qemu-system-i386 -kernel myOS.elf -serial pty & 运行。

使用 screen 命令进入交互界面（与 QEMU） sudo screen /dev/pts/x （x为生成的对应值）

第四部分 运行结果



可以看到，成功在uart和vga上显示。cmd help help cmd指令执行正常

右下角成功显示时间。